

RUNWAY MANAGEMENT SYSTEM

this project is based on runway management where multiple planes either land or take off in a controlled manner using multithread concept in C.

Coding:

```
#include<stdio.h>

#include<pthread.h>

#include<stdlib.h>

#include<unistd.h>

pthread_mutex_t runway;

int no_planes;

int flag=0;

void *rway(void *plane)

{

    int *plane_no = (int*)plane;

    if(*plane_no <= (no_planes)/2)

    {

        printf("plane %d is landing\n",*plane_no);

        pthread_mutex_lock(&runway);

        flag = 1;

        sleep(1);

        printf("plane %d is landed and runway is clear now\n",*plane_no);

        flag = 0;

        pthread_mutex_unlock(&runway);
```

```
}  
else  
{  
    printf("plane %d is ready to takeoff\n ",*plane_no);  
    while(flag)  
    {  
        sleep(2);  
    }  
    pthread_mutex_lock(&runway);  
    printf("plane %d is taking off\n",*plane_no);  
    sleep(1);  
    printf("plane %d is taken off and runway is clear now\n",*plane_no);  
    pthread_mutex_unlock(&runway);  
  
}  
pthread_exit(NULL);  
}  
  
int main()  
{  
    printf("Enter the no. of planes: ");  
    scanf("%d",&no_planes);  
    printf("\n");  
    pthread_t plane[no_planes];  
    int plane_id[no_planes];  
    pthread_mutex_init(&runway,NULL);
```

```
int itr=0;
for(itr = 0; itr < no_planes; itr++)
{
    plane_id[itr] = itr + 1;
    pthread_create(&plane[itr], NULL, rway, &plane_id[itr]);
    sleep(1);
}
for(itr = 0; itr < no_planes; itr++)
{
    pthread_join(plane[itr], NULL);
}
pthread_mutex_destroy(&runway);
exit(0);
}
```

Assumptions:

- The first half of the planes attempt to land.
- The remaining attempt to take off.

Output:

The screenshot displays the Visual Studio Code interface with a C program named `runway_management_system.c` open in the editor. The program is a multi-threaded application that simulates a runway management system. It includes headers for `stdio.h`, `pthread.h`, `stdlib.h`, and `unistd.h`. It defines a `pthread_mutex_t` named `runway`, an integer `no_planes`, and an integer `flag` initialized to 0. The `rway` function (likely a typo for `runway`) takes a `void *` `plane` as an argument. Inside the function, it casts `plane` to `int *` `plane_no`. It then checks if `*plane_no` is less than or equal to `(no_planes)/2`. If true, it prints "plane `%d` is landing\n", locks the `runway` mutex, sets `flag = 1`, sleeps for 1 second, and prints "plane `%d` is landed and runway is clear now\n".

The terminal output shows the execution of the program, displaying the following sequence of messages:

```
plane 4 is landed and runway is clear now
plane 5 is ready to takeoff
plane 5 is taking off
plane 5 is taken off and runway is clear now
plane 6 is ready to takeoff
plane 6 is taking off
plane 6 is taken off and runway is clear now
plane 7 is ready to takeoff
```

The status bar at the bottom indicates the current cursor position is at line 34, column 6, with 4 spaces. The encoding is UTF-8, and the line ending is CRLF. The window title is "Win32". The system tray shows the date and time as 10:58 AM on 31-03-2025.